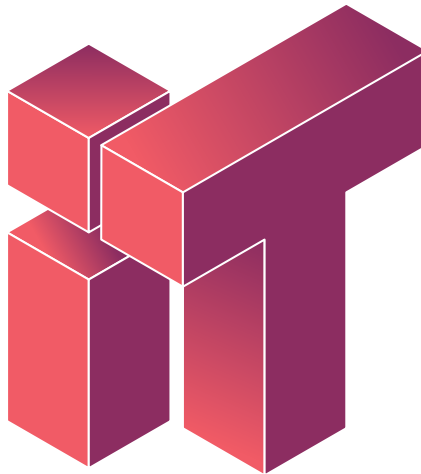


Data Parallelism with Rayon

Laboratory - Advanced Programming



INFOTRONICS

HES-SO Valais//Wallis

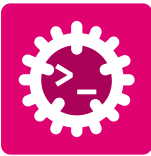
Systems Engineering • Infotronics

Advanced Programming • Silvan Zahno

Fall Semester 2026 • I3

Silvan Zahno, Rémy Borgeat

05.08.2026 • v1.0 • final



Contents

- 1 Goal 3
 - 1.1 What is Rayon? 3
 - 1.2 Setup 3
- 2 Refactor with Rayon 4
 - 2.1 Baseline 4
 - 2.2 Add Rayon to the Project 4
 - 2.3 count_humans / count_zombies 4
 - 2.4 tick_decay 4
 - 2.5 tick_infection 5
 - 2.6 tick_movement 5
 - 2.7 Benchmark 5
- 3 Checkout 6
- Glossary 7



1 | Goal

Modern CPUs have multiple cores, but most programs only use one. **Data parallelism** lets you harness all cores by processing chunks of data simultaneously.

In this laboratory you will refactor the Humans vs. Zombies simulation to use **Rayon**, Rust's data-parallelism library. You will learn to:

- Identify loops that can safely run in parallel
- Transform sequential iterators into parallel ones
- Recognise when a loop **cannot** be parallelised due to sequential dependencies
- Measure the speedup gained from parallel execution

1.1 What is Rayon?

Rayon turns sequential iterator chains into parallel ones. The API mirrors the standard iterator methods - you only need to add **par_**:

```
// Sequential
let sum: u32 = numbers.iter().map(|n| n * n).sum();

// Parallel (just add par_)
let sum: u32 = numbers.par_iter().map(|n| n * n).sum();
```

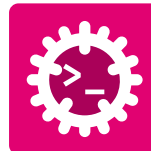
Rayon automatically splits your data across available CPU threads, processes chunks in parallel, and collects the results transparently.

The key requirement is that the data type must be **Send** (safe to transfer between threads) and **Sync** (safe to share via references). You will add these bounds to the **Entity** trait.

1.2 Setup

Clone the repository:

```
git clone <repo-url>
cd swd-course
```



2 | Refactor with Rayon

2.1 Baseline

```
cargo run --release -- bench
```



- Run **cargo run --release -- bench**.
- Note the averages – you will fill them in Table 1 at the end of the document.

2.2 Add Rayon to the Project

```
cargo add rayon
```



- Add **rayon** to **Cargo.toml**.
- Add **Send + Sync** to the **Entity** trait in **entity.rs**.
- Add **use rayon::prelude::*;** in **world.rs**.
- **cargo test** passes.

2.3 count_humans / count_zombies

Refactor both methods to parallelise them.



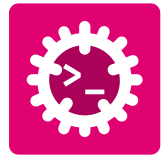
- Parallelise both methods.
- **cargo test** passes.

2.4 tick_decay

Refactor the **for** loop to parallelise it.



- Parallelise **tick_decay**.
- **cargo test** passes.



2.5 tick_infection

Parallelise the check phase (read-only). The removal and creation loops must stay sequential.



- Parallelise the infection check phase.
- **cargo test** passes.

2.6 tick_movement



Can **tick_movement** be parallelised with Rayon? Why or why not? What would happen if two entities computed their positions simultaneously?



- Understand why **tick_movement** stays sequential.
- All **cargo test** tests pass.

2.7 Benchmark

Run **bench** on your refactored version:

```
cargo run --release -- bench
```

Fill in your results and compare with your baseline:

Phase	Baseline (sequential)	Refactored (Rayon)	Δ
movement			
infection			
decay			
total			

Table 1 - Benchmark comparison (fill in your measurements)



Compare your measurements and answer:

1. Which phase benefits **most** from Rayon? Why?
2. Which phase gets **slower** with Rayon? Why?
3. Why doesn't movement change between the two versions?
4. What does this tell you about when parallelism is worth it?



3 | Checkout

Before finishing, ensure you have completed:

- ☐ Setup
 - ☐ **rayon** added to **Cargo.toml**
 - ☐ **Send + Sync** added to the **Entity** trait
 - ☐ **use rayon::prelude::*;** in **world.rs**
- ☐ Counting (**count_humans** / **count_zombies**)
 - ☐ Parallelised with Rayon
 - ☐ Tests pass
- ☐ Decay (**tick_decay**)
 - ☐ Parallelised with Rayon
 - ☐ Tests pass
- ☐ Infection (**tick_infection**)
 - ☐ Check phase parallelised with Rayon
 - ☐ Removal and creation loops stay sequential
 - ☐ Tests pass
- ☐ Movement (**tick_movement**)
 - ☐ Understand why it stays sequential
 - ☐ Tests pass
- ☐ Benchmark
 - ☐ Baseline recorded at start
 - ☐ Refactored version benchmarked
 - ☐ Table filled and analysis questions answered
- ☐ Final verification
 - ☐ **cargo test** reports all tests passing
 - ☐ **cargo run** shows the game running correctly



Glossary